

SCOMP ADC Peripheral for the DE10 Board

Kevin Ly

Submitted
April 28, 2026

Introduction

This document explains the design and implementation of the SCOMP ADC peripheral for the DE10 board, enabling the SCOMP processor to read real-world analog inputs through a memory-mapped interface. It is intended for future engineers who may extend or reuse the peripheral. Since SCOMP operates only on digital data, the peripheral interfaces with the LTC2308 ADC to convert analog voltages into digital values usable in software.

The primary design goal was to create a simple, reusable interface that abstracts ADC communication. Rather than managing serial timing and control signals, the software interacts with registers to select channels, initiate conversions, and read the result. Our approach prioritized software usability through a clean register-based interface, using software-controlled conversion triggering to give programs full control over data collection.

The final design met all project requirements and successfully demonstrated reliable real-time analog input processing across applications. Overall, we implemented a modular, software-driven peripheral with explicit conversion control, integrating it end-to-end with the SCOMP and validating it through working demonstrations.

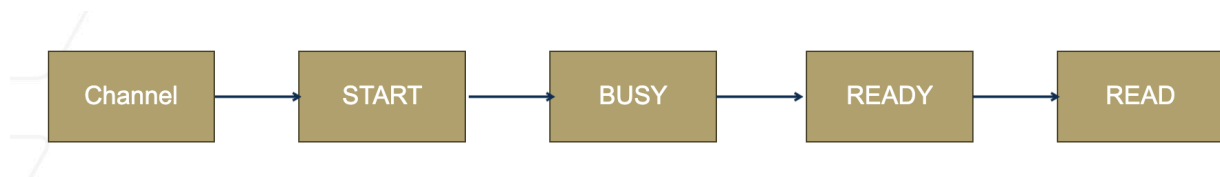


Figure 1. System Block Diagram

Device Functionality

ADC Interface Overview

The functionality of the ADC peripheral is based on 4 memory-mapped registers: control, status, data, and channel selection registers. These form a simple API for SCOMP programs to interact with the ADC. The peripheral follows a sequence: 1) write the desired input channel to the channel register, 2) a conversion is initiated by writing to the control register, 3) the program polls the status register until the conversion is complete, and 4) the result is read from the data register.

Address	Name	R/W	Purpose
0xC0	ADC_CTRL	W	Should start conversion and (optionally) clear the ready flag.
0xC1	ADC_STATUS	R	Reports if conversion is in progress and whether unread sampled data is available
0xC2	ADC_DATA	R	Returns the most recent conversion result as a 16-bit (by zero-extending) unsigned value.
0xC3	ADC_CHANNEL	R/W	Select which input channel will be sampled on next conversion & allows software to read back the current channel selection.

Figure 2. Register Map

Finite State Machine Operation

We also implemented a finite state machine within the peripheral to handle the ADC communication, managing conversion timing, serial data transfer, and result storage, allowing us to hide the low-level details and ensure accurate operation.

Status and Data Handling

The status register provides the flags: BUSY (indicates a conversion in progress) and READY (indicates valid data is available). This prevents invalid reads.

The data register returns the ADC result as a 12-bit value that is then zero extended to 16 bits to match SCOMP architecture, simplifying software handling or other complications.

The peripheral also supports multiple input channels, for flexibility in analog input usage. Because the conversions are triggered by software, the system can support continuous or event-driven sampling.

Functionality

To validate functionality, we developed two applications to demonstrate continuous real-time control and discrete input logic. The first is a target-matching game. The DE10 board displays a target value on its 7-segment displays, and the user adjusts a potentiometer to match it, with LEDs indicating whether the input is too high, too low, or within tolerance.

The second application is a two-step analog combination lock. The user adjusts two potentiometers across different channels until their voltages fall within predefined ranges, with each step requiring multiple consecutive stable readings before progressing, ultimately unlocking the system and displaying a success pattern.

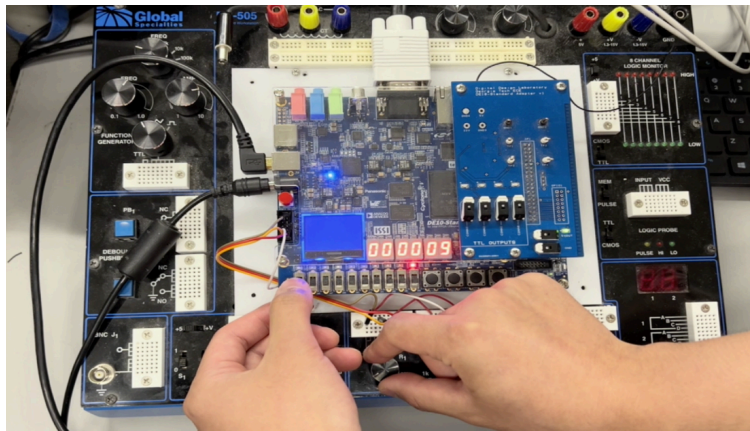


Figure 3. DE10 Board Programmed with Applications

Design Decisions and Implementation

Key design decisions included implementing a finite state machine to handle ADC communication: managing conversion timing, serial data transfer, and result storage, and retaining the 12-bit ADC output. We opted for software-controlled conversion triggering to give programs full control over data collection, and refined the register interface to include control, status, data, and channel registers, with a READY flag enabling reliable synchronization between software and hardware.

Conclusion

Our peripheral demonstrated that a register-based interface can effectively bridge analog hardware and a digital processor while keeping software simple and flexible. The software-controlled conversion approach proved effective during testing, allowing precise control over data collection. Future improvements include interrupt-driven operation to eliminate polling overhead and enhanced multi-channel sampling for efficiency. In retrospect, defining the register interface earlier would have sped up development, and more structured testing of ADC communication timing would have reduced synchronization issues.